SQL QUERIES

BASIC TO INTERMEDIATE

1. List all customers and the total number of orders they have placed. Show only customers with more than 5 orders. Sort by total orders descending.

```
select o.customerid, count(*) as total_orders from orders o
join customers c on c.customerid = o.customerid
group by o.customerid
having count(*) > 5
order by count(*) desc;
```

output:

	customerid	total_orders
1	SAVEA	31
2	ERNSH	30
3	QUICK	28
4	HUNGO	19
5	FOLKO	19
6	RATTC	18
7	HILAA	18
8	BERGS	18
9	BONAP	17
10	FRANK	15
11	WARTH	15
12	LEHMS	15
13	KOENE	14
14	HANAR	14
15	LILAS	14

2. Retrieve the total sales amount per customer by joining customers, orders, and order_details.
Show only customers whose total sales exceed 10,000. Sort by total sales descending.

```
select o.customerid, round(sum(unitprice * quantity * (1-discount)):: numeric, 2)
as total_sales from order_details od
join orders o on o.orderid = od.orderid
join customers c on c.customerid = o.customerid
group by o.customerid
having round(sum(unitprice * quantity * (1-discount))::numeric, 2) > 10000
order by sum(unitprice * quantity * (1-discount)) desc;
```

output:

	customerid	total_sales
1	QUICK	110,277.3
2	ERNSH	104,874.98
3	SAVEA	104,361.95
4	RATTC	51,097.8
5	HUNGO	49,979.9
6	HANAR	32,920.47
7	KOENE	30,908.38
8	FOLKO	29,567.56
9	MEREP	28,872.19
10	WHITC	27,363.6

3. Get the number of products per category.
Show only categories having more than 10 products. Sort by product count descending.

```
select p.categoryid, c.categoryname, count(*) as total_products from products p
join categories c on c.categoryid = p.categoryid
group by p.categoryid, c.categoryname
having count(*) > 10
```

```
order by count(*) desc;
```

output:

	123 categoryid	A-Z categoryname	123 total_products
1	1	Beverages	14
2	3	Confections	13
3	8	Seafood	12
4	2	Condiments	12

4. Display the total quantity sold per product.
Include only products where total quantity sold is greater than 100. Sort by quantity descending.

```
select od.productid, p.productname, sum(quantity) as total_quantity from
order_details od
join products p on p.productid = od.productid
group by od.productid, p.productname
having sum(quantity) > 100
order by sum(quantity) desc;
```

output:

	productid	productname	total_quantity
1	60	Camembert Pierrot	1,577
2	59	Raclette Courdavault	1,496
3	31	Gorgonzola Telino	1,397
4	56	Gnocchi di nonna Alice	1,263
5	16	Pavlova	1,158
6	75	Rhönbräu Klosterbier	1,155
7	24	Guaraná Fantástica	1,125
8	40	Boston Crab Meat	1,103
9	62	Tarte au sucre	1,083
10	2	Chang	1,069
11	71	Flotemysost	1,057
12	21	Sir Rodney's Scones	1,016
13	41	Jack's New England Clam Chowder	981
14	76	Lakkalikööri	981
15	17	Alice Mutton	978

5. Find the total number of orders handled by each employee.
Show only employees who handled more than 20 orders. Sort by order count descending.

```
select o.employeeid, concat(e.firstname, ' ', e.lastname) as employee_name,
count(*) as total_orders from orders o
join employees e on e.employeeid = o.employeeid
group by o.employeeid, concat(e.firstname, ' ', e.lastname)
having count(*) > 20
order by count(*);
```

output:

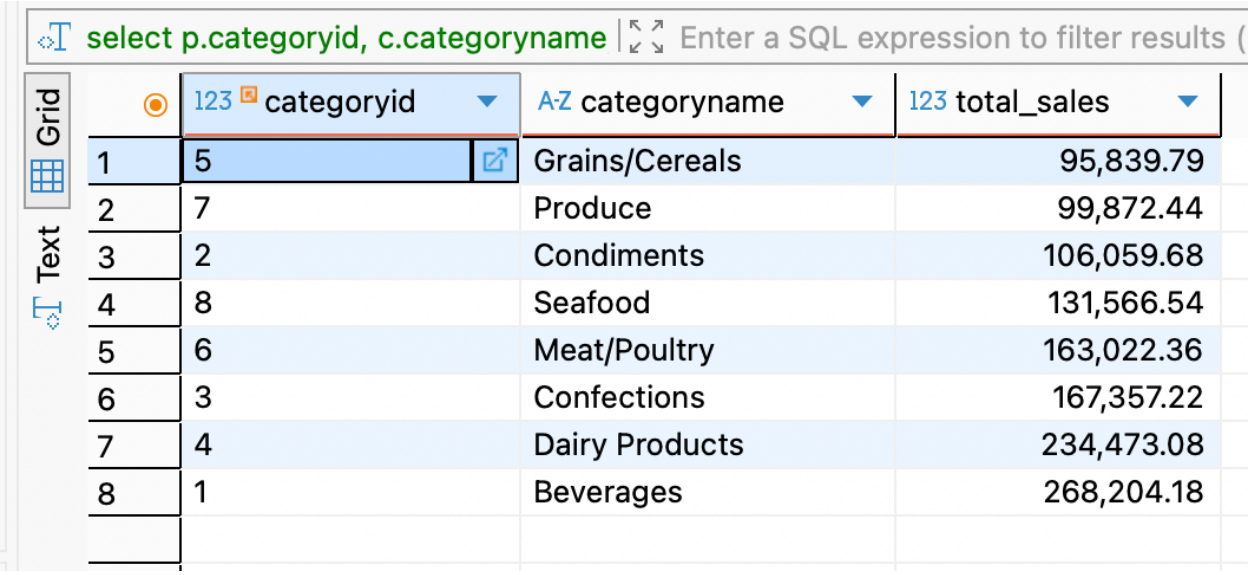
	employeeid	employee_name	total_orders
1	9	Anne Dodsworth	43
2	5	Steven Buchanan	44
3	6	Michael Suyama	67
4	7	Robert King	72
5	2	Andrew Fuller	98
6	8	Laura Callahan	104
7	1	Nancy Davolio	123
8	3	Janet Leverling	127
9	4	Margaret Peacock	156

INTERMEDIATE:

6. Retrieve the total sales per category by joining categories, products, and order_details.
Show only categories with total sales above 50,000. Sort by total sales descending.

```
select p.categoryid, c.categoryname, round(sum(od.unitprice * od.quantity * (1-
od.discount))::numeric, 2) as total_sales from order_details od
join products p on p.productid = od.productid
join categories c on c.categoryid = p.categoryid
group by p.categoryid, c.categoryname
having round(sum(od.unitprice * od.quantity * (1-od.discount))::numeric, 2) >
50000
order by round(sum(od.unitprice * od.quantity * (1-od.discount))::numeric, 2);
```

Output:

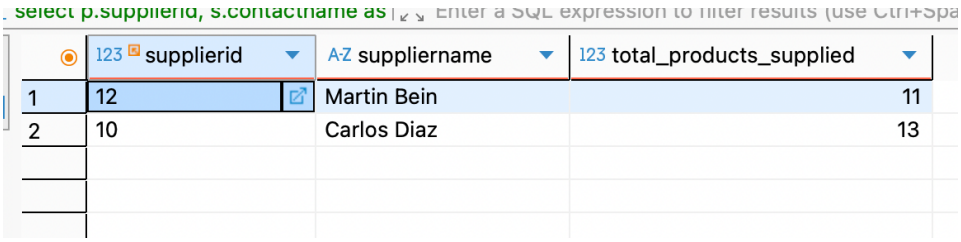


	categoryid	categoryname	total_sales
1	5	Grains/Cereals	95,839.79
2	7	Produce	99,872.44
3	2	Condiments	106,059.68
4	8	Seafood	131,566.54
5	6	Meat/Poultry	163,022.36
6	3	Confections	167,357.22
7	4	Dairy Products	234,473.08
8	1	Beverages	268,204.18

7. List suppliers and the number of products they supply.
Show only suppliers who supply more than 5 products. Sort by product count descending.

```
select p.supplierid, s.contactname as suppliername, count(*) as
total_products_supplied from products p
join suppliers s on s.supplierid = p.supplierid
group by p.supplierid, s.contactname
having count(*) > 5
order by count(*);
```

output:

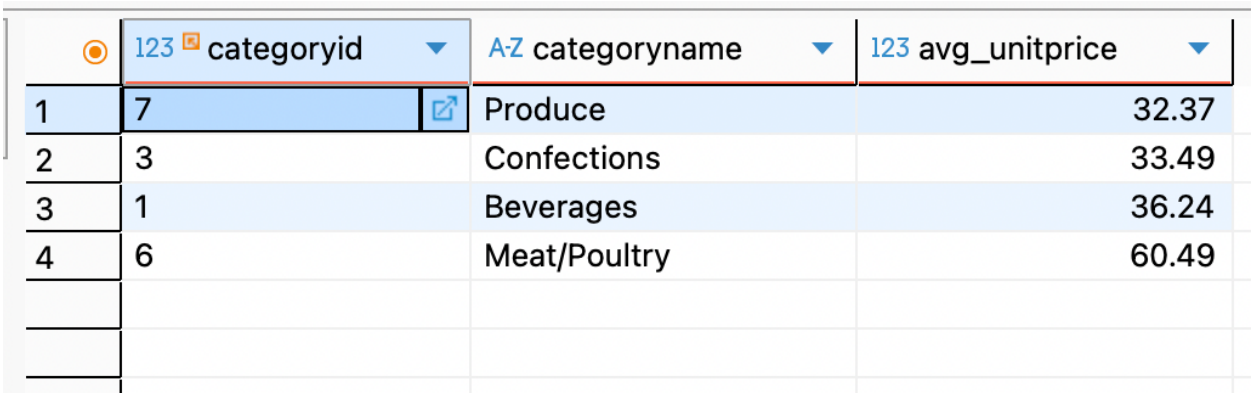


	supplierid	suppliername	total_products_supplied
1	12	Martin Bein	11
2	10	Carlos Diaz	13

8. Get the average unit price per category.
Show only categories where the average price is above 30. Sort by average price descending.

```
select p.categoryid, c.categoryname, round(avg(p.unitprice)::numeric, 2) as
avg_unitprice from products p
join categories c on c.categoryid = p.categoryid
group by p.categoryid, c.categoryname
having round(avg(p.unitprice)::numeric, 2) > 30
order by round(avg(p.unitprice)::numeric, 2);
```

output:



	123 categoryid	A-Z categoryname	123 avg_unitprice
1	7	Produce	32.37
2	3	Confections	33.49
3	1	Beverages	36.24
4	6	Meat/Poultry	60.49

9. Display the total revenue generated per employee (orders + order_details).
Show only employees generating more than 20,000 in revenue. Sort by revenue descending.

```
select o.employeeid, concat(e.firstname, ' ', e.lastname) as employee_name,
round(sum(od.unitprice * od.quantity * (1-discount))::numeric, 2) as
total_revenue_by_employee from order_details od
join orders o on o.orderid = od.orderid
join employees e on e.employeeid = o.employeeid
group by o.employeeid, concat(e.firstname, ' ', e.lastname)
having  round(sum(od.unitprice * od.quantity * (1-discount))::numeric, 2) >
20000
order by round(sum(od.unitprice * od.quantity * (1-discount))::numeric, 2) desc;
```

output:

	employeeid	employee_name	total_revenue_by_employee
1	4	Margaret Peacock	232,969.95
2	3	Janet Leverling	202,812.84
3	1	Nancy Davolio	192,107.6
4	2	Andrew Fuller	167,159.25
5	8	Laura Callahan	126,862.28
6	7	Robert King	124,568.23
7	9	Anne Dodsworth	77,308.07
8	6	Michael Suyama	73,726.79
9	5	Steven Buchanan	68,785.28

10. Retrieve the number of orders shipped to each country. Show only countries with more than 10 orders. Sort by order count descending.

```
select shipcountry, count(*) as total_orders_shipped from orders
group by shipcountry
having count(*) > 10
order by count(*);
```

output:

	shipcountry	total_orders_shipped
1	Portugal	13
2	Argentina	16
3	Switzerland	18
4	Denmark	18
5	Ireland	19
6	Belgium	19
7	Finland	22
8	Spain	23
9	Italy	28
10	Canada	30
11	Mexico	30
12	Sweden	37
13	Austria	40
14	Venezuela	46
15	UK	56

ADVANCED:

11. Find customers and the average order value (orders + order_details). Show only customers with average order value greater than 500. Sort by average descending.

```
select
   order_totals.customerid,
   round(avg(order_total)::numeric, 2) as avg_order_value
from (
   select
      o.orderid,
      o.customerid,
      sum(od.unitprice * od.quantity * (1 - od.discount)) as order_total
   from orders o
   join order_details od on o.orderid = od.orderid
   group by o.orderid, o.customerid
) as order_totals
join customers c on c.customerid = order_totals.customerid
group by order_totals.customerid
having round(avg(order_total)::numeric, 2) > 500
order by avg_order_value desc;
```

output:

	customerid	avg_order_value
1	QUICK	3,938.48
2	ERNSH	3,495.83
3	SAVEA	3,366.51
4	RATTC	2,838.77
5	HUNGO	2,630.52
6	SIMOB	2,402.44
7	HANAR	2,351.46
8	FOLIG	2,333.38
9	PICCO	2,312.89
10	MEREP	2,220.94

12. Get the top-selling products per category (by total quantity sold).
Show only products with total quantity sold above 200. Sort within category by
quantity descending.

```
select
    p.categoryid,
    c.categoryname,
    p.productid,
    p.productname,
    sum(od.quantity) as total_quantity
from order_details od
join products p on p.productid = od.productid
join categories c on c.categoryid = p.categoryid
group by p.categoryid, c.categoryname, p.productid, p.productname
having sum(od.quantity) > 200
order by p.categoryid, total_quantity desc;
```

output:

	categoryid	categoryname	productid	productname	total_quantity
1	1	Beverages	75	Rhönbräu Klosterbier	1,155
2	1	Beverages	24	Guaraná Fantástica	1,125
3	1	Beverages	2	Chang	1,069
4	1	Beverages	76	Lakkalikööri	981
5	1	Beverages	35	Steeleye Stout	883
6	1	Beverages	1	Chai	834
7	1	Beverages	70	Outback Lager	817
8	1	Beverages	39	Chartreuse verte	793
9	1	Beverages	38	Côte de Blaye	623
10	1	Beverages	43	Ipoh Coffee	580
11	1	Beverages	34	Sasquatch Ale	506
12	2	Condiments	77	Original Frankfurter grüne Soße	791
13	2	Condiments	65	Louisiana Fiery Hot Pepper Sauce	745
14	2	Condiments	61	Sirop d'érable	603
15	2	Condiments	44	Gula Malacca	601
16	2	Condiments	4	Chef Anton's Cajun Seasoning	453
17	2	Condiments	63	Vegie-spread	445
18	2	Condiments	8	Northwoods Cranberry Sauce	372
19	2	Condiments	3	Aniseed Syrup	328

**13. Retrieve the total discount given per product (order_details).
Show only products where total discount exceeds 1,000. Sort by discount descending.**

```
select productid, round(sum(unitprice * quantity * discount)::numeric, 2) as
total_discount_amount from order_details
group by productid
having round(sum(unitprice * quantity * discount)::numeric, 2) > 1000
order by total_discount_amount;
```

output:

	productid	total_discount_amount
8	31	1,251.63
9	10	1,272.86
10	71	1,325.48
11	30	1,351.34
12	1	1,489.5
13	16	1,532.27
14	43	1,552.5
15	9	1,600.5
16	26	1,685.76
17	55	2,085.6
18	61	2,086.2
19	2	2,203.24
20	69	2,364.84

14. List employees and the number of unique customers they handled. Show only employees who handled more than 15 unique customers. Sort by count descending.

```
select e.employeeid, concat(e.firstname, ' ', e.lastname) as employeename,
count(distinct o.customerid) as unique_customers_handled
from orders o
join employees e on e.employeeid = o.employeeid
group by e.employeeid, e.firstname , e.lastname
having count(distinct o.customerid ) > 15
order by unique_customers_handled desc
```

output:

	123 employeeid	A-Z employeename	123 unique_customers_handled
1	4	Margaret Peacock	75
2	1	Nancy Davolio	65
3	3	Janet Leverling	63
4	2	Andrew Fuller	60
5	8	Laura Callahan	56
6	7	Robert King	45
7	6	Michael Suyama	43
8	5	Steven Buchanan	31
9	9	Anne Dodsworth	29

15. Find the monthly total sales (year + month) using orders and order_details.
Show only months where total sales exceed 30,000. Sort by year and month ascending.

```
select extract(year from o.orderdate) as year,
extract(month from o.orderdate) as month,
round(sum(od.unitprice * od.quantity * (1-discount))::numeric, 2) as total_sales
from orders o join order_details od
on o.orderid = od.orderid
group by extract(year from o.orderdate),
extract(month from o.orderdate)
having round(sum(od.unitprice * od.quantity * (1-discount))::numeric, 2) >
30000
order by year, month;
```

output:

	123 year	123 month	123 total_sales
1	1,996	10	37,515.72
2	1,996	11	45,600.04
3	1,996	12	45,239.63
4	1,997	1	61,258.07
5	1,997	2	38,483.63
6	1,997	3	38,547.22
7	1,997	4	53,032.95
8	1,997	5	53,781.29
9	1,997	6	36,362.8
10	1,997	7	51,020.86

STORED PROCEDURES:

1. Create a stored procedure to insert a new customer.

```
create or replace procedure insert_customer(
customerid varchar(5),
companyname varchar(40),
contactname varchar(30)
)
language plpgsql
as $$
begin
        if customerid is null or companyname is null or contactname is null then
raise exception 'customerid, companyname, contactname is required';
rollback;
end if;

insert into customers(customerid, companyname, contactname) values
(customerid, companyname, contactname);
commit;
```

```
raise notice 'customer % inserted successfully', customerid;

end;
$$;

call insert_customer('AS001', 'Alia', 'ABC Corp');
call insert_customer('AS002', 'Blia', 'ABC Corp');

select * from customers;
```

output:
customer AS001 inserted successfully
customer AS002 inserted successfully

Table: customers

	customerid	yname	contactname	contacttitle	address	city	region
84	VICTE	Victuailles en stock	Mary Saveley	Sales Agent	2, rue du Commerce	Lyon	[NULL]
85	VINET	Vins et alcools Chevalier	Paul Henriot	Accounting Manager	59 rue de l'Abbaye	Reims	[NULL]
86	WANDK	Die Wandernde Kuh	Rita Müller	Sales Representative	Adenauerallee 900	Stuttgart	[NULL]
87	WARTH	Wartian Herkku	Pirkko Koskitalo	Accounting Manager	Torikatu 38	Oulu	[NULL]
88	WELLI	Wellington Importadora	Paula Parente	Sales Manager	Rua do Mercado, 12	Resende	SP
89	WHITC	White Clover Markets	Karl Jablonski	Owner	305 - 14th Ave. S. Suite 3B	Seattle	WA
90	WILMK	Wilman Kala	Matti Karttunen	Owner/Marketing Assis	Keskuskatu 45	Helsinki	[NULL]
91	WOLZA	Wolski Zajazd	Zbyszek Piestrzeniewicz	Owner	ul. Filtrowa 68	Warszawa	[NULL]
92	AS001	Alia	ABC Corp	[NULL]	[NULL]	[NULL]	[NULL]
93	AS002	Blia	ABC Corp	[NULL]	[NULL]	[NULL]	[NULL]

2. Create a stored procedure to place a new order for an existing customer with one product. Insert into orders and order_details in a single transaction.

```
create or replace procedure place_new_order( p_customerid varchar(50),
p_productid int, p_quantity int

)

language plpgsql

as $$

declare

   v_orderid int;
```

```
    v_unitprice dec;
    v_discount float := 0;
begin
    if not exists (select 1 from customers where customerid = p_customerid)
then
        raise exception 'customer with id % does not exist', p_customerid;
rollback;
    end if;
    select unitprice into v_unitprice from products
    where productid = p_productid;
    if not found then
        raise exception 'product with id % does not exist', p_productid;
rollback;
    end if;
    if p_quantity <= 0 then
        raise exception 'quantity must be greater than 0';
rollback;
    end if;
    insert into orders(
        customerid,
        orderdate,
        requireddate,
```

```
    shipcountry
) values (
    p_customerid,
    current_date,
    current_date + interval '7 days',
    (select country from customers where customerid = p_customerid)
)
returning orderid into v_orderid;
insert into order_details(
    orderid,
    productid,
    unitprice,
    quantity,
    discount
) values (
    v_orderid,
    p_productid,
    v_unitprice,
    p_quantity,
    v_discount
);
```

```
    raise notice 'order placed successfully! order id: %, product: %, quantity: %, total = %',
        v_orderid, p_productid, p_quantity, (v_unitprice * p_quantity);
end;
$$;
```

```
call place_new_order('anatr', 2, 5);
```

output:

Order placed successfully! Order ID: 11086, Product: 2, Quantity: 5, Total = 95.0000

Table: orders

	orderid	productid	unitprice	quantity	discount
2132	11,077	3	10	4	0
2133	11,077	4	22	1	0
2134	11,077	6	25	1	0.02
2135	11,077	7	30	1	0.05
2136	11,077	8	40	2	0.1
2137	11,077	10	31	1	0
2138	11,077	12	38	2	0.05
2139	11,077	13	6	4	0
2140	11,077	14	23.25	1	0.03
2141	11,077	16	17.45	2	0.03
2142	11,077	20	81	1	0.04
2143	11,077	23	9	2	0
2144	11,077	32	32	1	0
2145	11,077	39	18	2	0.05
2146	11,077	41	9.65	3	0
2147	11,077	46	12	3	0.02
2148	11,077	52	7	2	0
2149	11,077	55	24	2	0
2150	11,077	60	34	2	0.06
2151	11,077	64	33.25	2	0.03
2152	11,077	66	17	1	0
2153	11,077	73	15	2	0.01
2154	11,077	75	7.75	4	0
2155	11,077	77	13	2	0
2156	11,078	1	18	1	0
2157	11,085	2	19	1	0
2158	11,086	2	19	5	0
2159	11,087	2	19	5	0

3. Create a stored procedure to update product stock after an order is placed. If stock is not enough, rollback the transaction.

```
create or replace procedure update_stock(
    p_order_id int
)
language plpgsql
as $$
declare
    v_product record;
begin
    for v_product in
        select productid, quantity from order_details where orderid = p_order_id
    loop
        update products
        set unitsinstock = unitsinstock - v_product.quantity
        where productid = v_product.productid
          and unitsinstock >= v_product.quantity;

        if not found then
            raise exception 'insufficient stock for product id: %', v_product.productid;
rollback;
```

```
        end if;
    end loop;


    commit;
    raise notice 'stock updated for order %', p_order_id;
end;
$$;
```

Table: products(before order placed and stock updation -> for productid=2, unitsinstock=11

	123 prodı	A-Z productname	123 supplier	123	A-Z quanti	123 unitp	123 unitsinstock	123 unitsonorder	123 reorderlevel	123 dis
50	52	Filo Mix	24	5	16 - 2 kg boxe	7	38	0	25	
51	53	Perth Pasties	24	6	48 pieces	32.8	0	0	0	
52	54	Tourtière	25	6	16 pies	7.45	21	0	10	
53	55	Pâté chinois	25	6	24 boxes x 2	24	115	0	20	
54	56	Gnocchi di nonna Alice	26	5	24 - 250 g pk	38	21	10	30	
55	57	Ravioli Angelo	26	5	24 - 250 g pk	19.5	36	0	20	
56	58	Escargots de Bourgogr	27	8	24 pieces	13.25	62	0	20	
57	59	Raclette Courdavault	28	4	5 kg pkg.	55	79	0	0	
58	60	Camembert Pierrot	28	4	15 - 300 g rou	34	19	0	0	
59	61	Sirop d'érable	29	2	24 - 500 ml b	28.5	113	0	25	
60	62	Tarte au sucre	29	3	48 pies	49.3	17	0	0	
61	63	Vegie-spread	7	2	15 - 625 g jar	43.9	24	0	5	
62	64	Wimmers gute Semmel	12	5	20 bags x 4 p	33.25	22	80	30	
63	65	Louisiana Fiery Hot Pep	2	2	32 - 8 oz bott	21.05	76	0	0	
64	66	Louisiana Hot Spiced C	2	2	24 - 8 oz jars	17	4	100	20	
65	67	Laughing Lumberjack L	16	1	24 - 12 oz bot	14	52	0	10	
66	68	Scottish Longbreads	8	3	10 boxes x 8 p	12.5	6	10	15	
67	69	Gudbrandsdalsost	15	4	10 kg pkg.	36	26	0	15	
68	70	Outback Lager	7	1	24 - 355 ml b	15	15	10	30	
69	71	Flotemysost	15	4	10 - 500 g pk	21.5	26	0	0	
70	72	Mozzarella di Giovanni	14	4	24 - 200 g pk	34.8	14	0	0	
71	73	Röd Kaviar	17	8	24 - 150 g jar	15	101	0	5	
72	74	Longlife Tofu	4	7	5 kg pkg.	10	4	20	5	
73	75	Rhönbräu Klosterbier	12	1	24 - 0.5 l bott	7.75	125	0	25	
74	76	Lakkalikööri	23	1	500 ml	18	57	0	20	
75	77	Original Frankfurter grü	12	2	12 boxes	13	32	0	15	
76	1	Chai	1	1	10 boxes x 20	18	38	0	10	
77	2	Chang	1	1	24 - 12 oz bot	19	11	40	25	

```
call update_stock(11087);
```

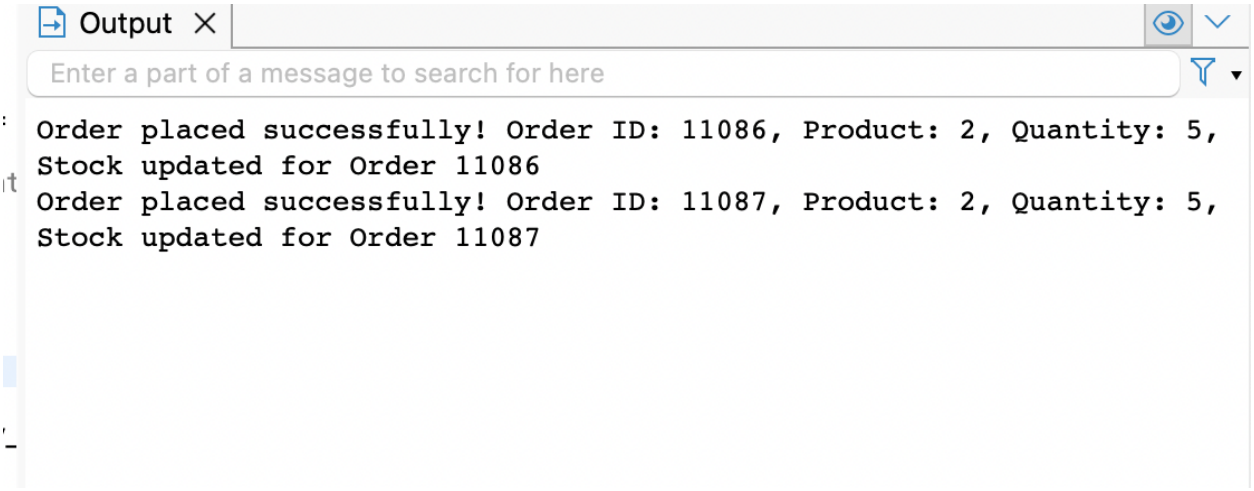


Table: orders

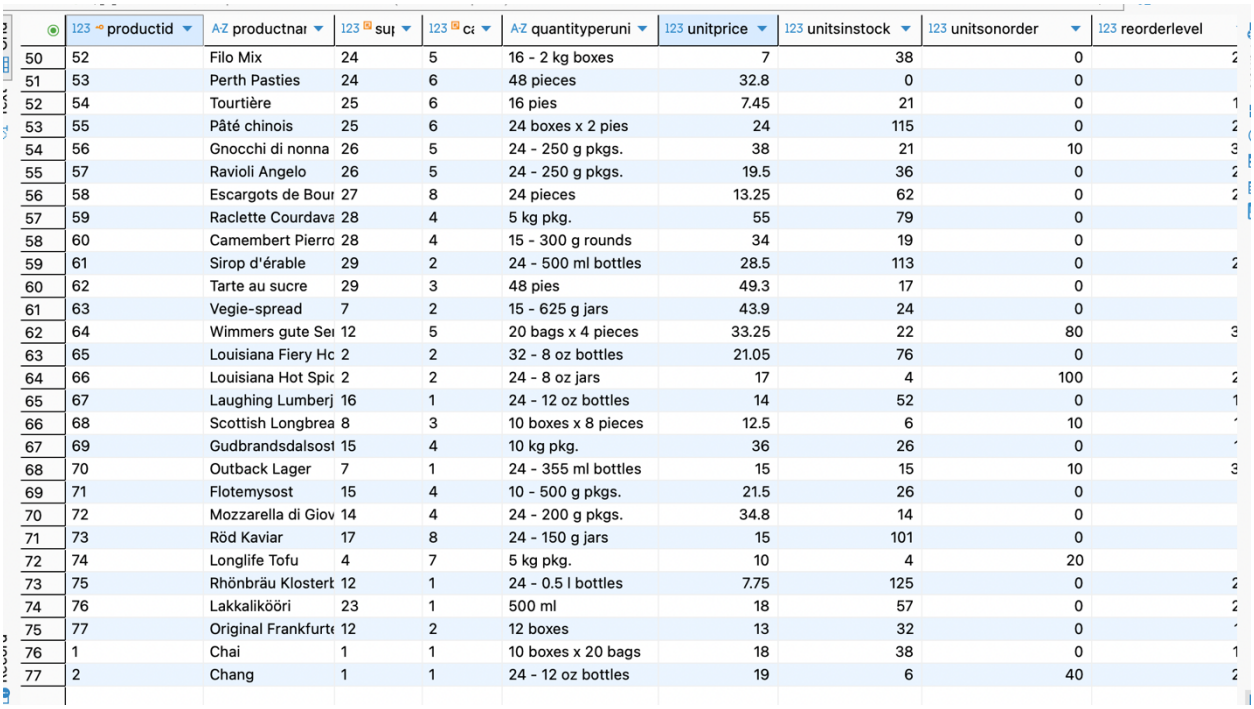
Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	orderid	customerid	employeeid	orderdate	requireddate	shippeddate	shipvia
807	11,054	CACTU	8	1998-04-28 00:00:00.000	1998-05-26 00:00:00.000	[NULL]	1
808	11,055	HILAA	7	1998-04-28 00:00:00.000	1998-05-26 00:00:00.000	1998-05-05 00:00:00.000	2
809	11,056	EASTC	8	1998-04-28 00:00:00.000	1998-05-12 00:00:00.000	1998-05-01 00:00:00.000	2
810	11,057	NORTS	3	1998-04-29 00:00:00.000	1998-05-27 00:00:00.000	1998-05-01 00:00:00.000	3
811	11,058	BLAUS	9	1998-04-29 00:00:00.000	1998-05-27 00:00:00.000	[NULL]	3
812	11,059	RICAR	2	1998-04-29 00:00:00.000	1998-06-10 00:00:00.000	[NULL]	2
813	11,060	FRANS	2	1998-04-30 00:00:00.000	1998-05-28 00:00:00.000	1998-05-04 00:00:00.000	2
814	11,061	GREAL	4	1998-04-30 00:00:00.000	1998-06-11 00:00:00.000	[NULL]	3
815	11,062	REGGC	4	1998-04-30 00:00:00.000	1998-05-28 00:00:00.000	[NULL]	2
816	11,063	HUNGO	3	1998-04-30 00:00:00.000	1998-05-28 00:00:00.000	1998-05-06 00:00:00.000	2
817	11,064	SAVEA	1	1998-05-01 00:00:00.000	1998-05-29 00:00:00.000	1998-05-04 00:00:00.000	1
818	11,065	LILAS	8	1998-05-01 00:00:00.000	1998-05-29 00:00:00.000	[NULL]	1
819	11,066	WHITC	7	1998-05-01 00:00:00.000	1998-05-29 00:00:00.000	1998-05-04 00:00:00.000	2
820	11,067	DRACD	1	1998-05-04 00:00:00.000	1998-05-18 00:00:00.000	1998-05-06 00:00:00.000	2
821	11,068	QUEEN	8	1998-05-04 00:00:00.000	1998-06-01 00:00:00.000	[NULL]	2
822	11,069	TORTU	1	1998-05-04 00:00:00.000	1998-06-01 00:00:00.000	1998-05-06 00:00:00.000	2
823	11,070	LEHMS	2	1998-05-05 00:00:00.000	1998-06-02 00:00:00.000	[NULL]	1
824	11,071	LILAS	1	1998-05-05 00:00:00.000	1998-06-02 00:00:00.000	[NULL]	1
825	11,072	ERNSH	4	1998-05-05 00:00:00.000	1998-06-02 00:00:00.000	[NULL]	2
826	11,073	PERIC	2	1998-05-05 00:00:00.000	1998-06-02 00:00:00.000	[NULL]	2
827	11,074	SIMOB	7	1998-05-06 00:00:00.000	1998-06-03 00:00:00.000	[NULL]	2
828	11,075	RICSU	8	1998-05-06 00:00:00.000	1998-06-03 00:00:00.000	[NULL]	2
829	11,076	BONAP	4	1998-05-06 00:00:00.000	1998-06-03 00:00:00.000	[NULL]	2
830	11,077	RATTC	1	1998-05-06 00:00:00.000	1998-06-03 00:00:00.000	[NULL]	2
831	11,078	ALFKI	5	2026-04-29 00:00:00.000	2026-05-06 00:00:00.000	[NULL]	[NULL]
832	11,085	ANATR	5	2026-04-29 00:00:00.000	2026-05-06 00:00:00.000	[NULL]	[NULL]
833	11,086	ANATR	[NULL]	2026-04-29 00:00:00.000	2026-05-06 00:00:00.000	[NULL]	[NULL]
834	11,087	ANATR	[NULL]	2026-04-29 00:00:00.000	2026-05-06 00:00:00.000	[NULL]	[NULL]

Table: order_details

	orderid	productid	unitprice	quantity	discount
2132	11,077	3	10	4	0
2133	11,077	4	22	1	0
2134	11,077	6	25	1	0.02
2135	11,077	7	30	1	0.05
2136	11,077	8	40	2	0.1
2137	11,077	10	31	1	0
2138	11,077	12	38	2	0.05
2139	11,077	13	6	4	0
2140	11,077	14	23.25	1	0.03
2141	11,077	16	17.45	2	0.03
2142	11,077	20	81	1	0.04
2143	11,077	23	9	2	0
2144	11,077	32	32	1	0
2145	11,077	39	18	2	0.05
2146	11,077	41	9.65	3	0
2147	11,077	46	12	3	0.02
2148	11,077	52	7	2	0
2149	11,077	55	24	2	0
2150	11,077	60	34	2	0.06
2151	11,077	64	33.25	2	0.03
2152	11,077	66	17	1	0
2153	11,077	73	15	2	0.01
2154	11,077	75	7.75	4	0
2155	11,077	77	13	2	0
2156	11,078	1	18	1	0
2157	11,085	2	19	1	0
2158	11,086	2	19	5	0
2159	11,087	2	19	5	0

Table: products(after order placed and stock updation -> for productid=2, unitsinstock=11-5=6



	productid	productnar	sup	ca	quantityperuni	unitprice	unitsinstock	unitsonorder	reorderlevel
50	52	Filo Mix	24	5	16 - 2 kg boxes	7	38	0	2
51	53	Perth Pasties	24	6	48 pieces	32.8	0	0	
52	54	Tourtière	25	6	16 pies	7.45	21	0	1
53	55	Pâté chinois	25	6	24 boxes x 2 pies	24	115	0	2
54	56	Gnocchi di nonna	26	5	24 - 250 g pkgs.	38	21	10	3
55	57	Ravioli Angelo	26	5	24 - 250 g pkgs.	19.5	36	0	2
56	58	Escargots de Bour	27	8	24 pieces	13.25	62	0	2
57	59	Raclette Courdava	28	4	5 kg pkg.	55	79	0	
58	60	Camembert Pierro	28	4	15 - 300 g rounds	34	19	0	
59	61	Sirop d'érable	29	2	24 - 500 ml bottles	28.5	113	0	2
60	62	Tarte au sucre	29	3	48 pies	49.3	17	0	
61	63	Vegie-spread	7	2	15 - 625 g jars	43.9	24	0	
62	64	Wimmers gute Sei	12	5	20 bags x 4 pieces	33.25	22	80	3
63	65	Louisiana Fiery Hc	2	2	32 - 8 oz bottles	21.05	76	0	
64	66	Louisiana Hot Spic	2	2	24 - 8 oz jars	17	4	100	2
65	67	Laughing Lumberj	16	1	24 - 12 oz bottles	14	52	0	1
66	68	Scottish Longbrea	8	3	10 boxes x 8 pieces	12.5	6	10	
67	69	Gudbrandsdalsost	15	4	10 kg pkg.	36	26	0	
68	70	Outback Lager	7	1	24 - 355 ml bottles	15	15	10	3
69	71	Flotemysost	15	4	10 - 500 g pkgs.	21.5	26	0	
70	72	Mozzarella di Giov	14	4	24 - 200 g pkgs.	34.8	14	0	
71	73	Röd Kaviar	17	8	24 - 150 g jars	15	101	0	
72	74	Longlife Tofu	4	7	5 kg pkg.	10	4	20	
73	75	Rhönbräu Klosterb	12	1	24 - 0.5 l bottles	7.75	125	0	2
74	76	Lakkalikööri	23	1	500 ml	18	57	0	2
75	77	Original Frankfurte	12	2	12 boxes	13	32	0	
76	1	Chai	1	1	10 boxes x 20 bags	18	38	0	1
77	2	Chang	1	1	24 - 12 oz bottles	19	6	40	2

4. Create a stored procedure to cancel an order. Delete records from order_details first, then from orders, using a transaction.

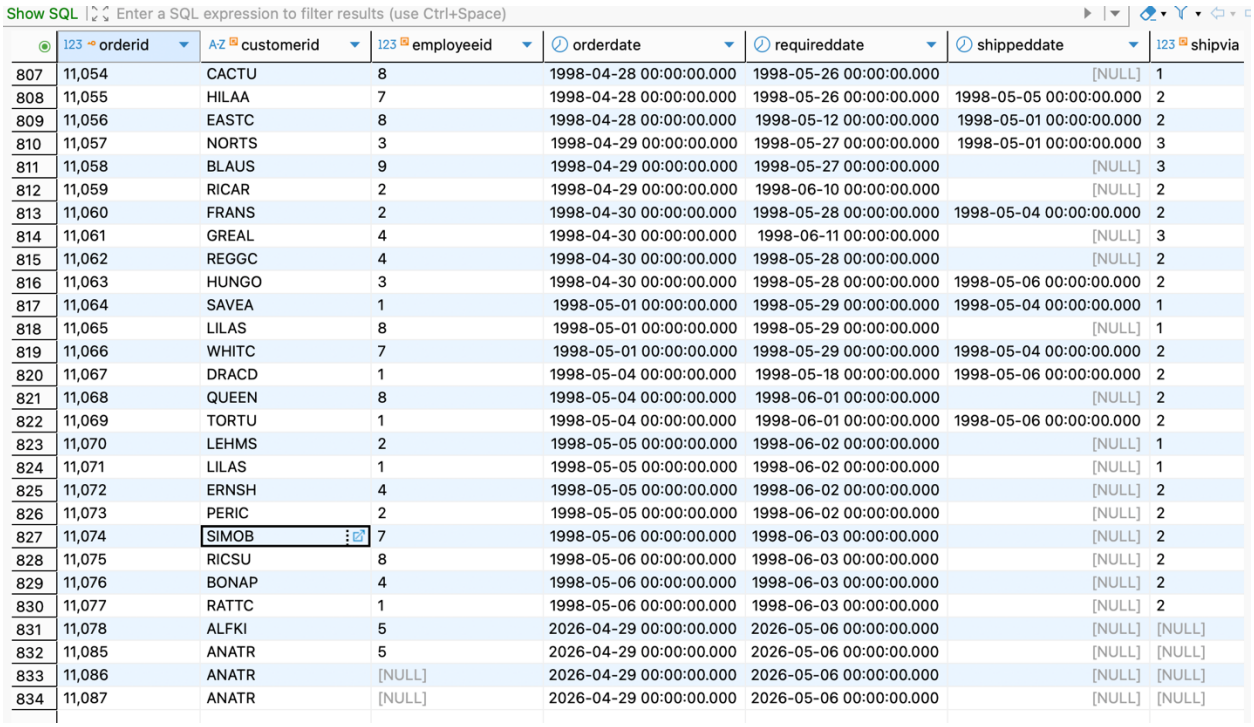
```
create or replace procedure cancel_order(
p_orderid int
)
language plpgsql
as $$
begin
        if not exists (select 1 from orders where orderid=p_orderid)
        then
        raise exception 'order % does not exist', p_orderid;
rollback;
end if;
delete from order_details where orderid = p_orderid;
delete from orders where orderid = p_orderid;
commit;
```

```
raise notice 'order % cancelled successfully', p_orderid;
end;
$$;

call cancel_order(11087);
```

output:
order 11087 cancelled successfully

table: order (before cancelling order=11087)



Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	orderid	customerid	employeeid	orderdate	requireddate	shippeddate	shipvia
807	11,054	CACTU	8	1998-04-28 00:00:00.000	1998-05-26 00:00:00.000	[NULL]	1
808	11,055	HILAA	7	1998-04-28 00:00:00.000	1998-05-26 00:00:00.000	1998-05-05 00:00:00.000	2
809	11,056	EASTC	8	1998-04-28 00:00:00.000	1998-05-12 00:00:00.000	1998-05-01 00:00:00.000	2
810	11,057	NORTS	3	1998-04-29 00:00:00.000	1998-05-27 00:00:00.000	1998-05-01 00:00:00.000	3
811	11,058	BLAUS	9	1998-04-29 00:00:00.000	1998-05-27 00:00:00.000	[NULL]	3
812	11,059	RICAR	2	1998-04-29 00:00:00.000	1998-06-10 00:00:00.000	[NULL]	2
813	11,060	FRANS	2	1998-04-30 00:00:00.000	1998-05-28 00:00:00.000	1998-05-04 00:00:00.000	2
814	11,061	GREAL	4	1998-04-30 00:00:00.000	1998-06-11 00:00:00.000	[NULL]	3
815	11,062	REGGC	4	1998-04-30 00:00:00.000	1998-05-28 00:00:00.000	[NULL]	2
816	11,063	HUNGO	3	1998-04-30 00:00:00.000	1998-05-28 00:00:00.000	1998-05-06 00:00:00.000	2
817	11,064	SAVEA	1	1998-05-01 00:00:00.000	1998-05-29 00:00:00.000	1998-05-04 00:00:00.000	1
818	11,065	LILAS	8	1998-05-01 00:00:00.000	1998-05-29 00:00:00.000	[NULL]	1
819	11,066	WHITC	7	1998-05-01 00:00:00.000	1998-05-29 00:00:00.000	1998-05-04 00:00:00.000	2
820	11,067	DRACD	1	1998-05-04 00:00:00.000	1998-05-18 00:00:00.000	1998-05-06 00:00:00.000	2
821	11,068	QUEEN	8	1998-05-04 00:00:00.000	1998-06-01 00:00:00.000	[NULL]	2
822	11,069	TORTU	1	1998-05-04 00:00:00.000	1998-06-01 00:00:00.000	1998-05-06 00:00:00.000	2
823	11,070	LEHMS	2	1998-05-05 00:00:00.000	1998-06-02 00:00:00.000	[NULL]	1
824	11,071	LILAS	1	1998-05-05 00:00:00.000	1998-06-02 00:00:00.000	[NULL]	1
825	11,072	ERNSH	4	1998-05-05 00:00:00.000	1998-06-02 00:00:00.000	[NULL]	2
826	11,073	PERIC	2	1998-05-05 00:00:00.000	1998-06-02 00:00:00.000	[NULL]	2
827	11,074	SIMOB	7	1998-05-06 00:00:00.000	1998-06-03 00:00:00.000	[NULL]	2
828	11,075	RICSU	8	1998-05-06 00:00:00.000	1998-06-03 00:00:00.000	[NULL]	2
829	11,076	BONAP	4	1998-05-06 00:00:00.000	1998-06-03 00:00:00.000	[NULL]	2
830	11,077	RATTC	1	1998-05-06 00:00:00.000	1998-06-03 00:00:00.000	[NULL]	2
831	11,078	ALFKI	5	2026-04-29 00:00:00.000	2026-05-06 00:00:00.000	[NULL]	[NULL]
832	11,085	ANATR	5	2026-04-29 00:00:00.000	2026-05-06 00:00:00.000	[NULL]	[NULL]
833	11,086	ANATR	[NULL]	2026-04-29 00:00:00.000	2026-05-06 00:00:00.000	[NULL]	[NULL]
834	11,087	ANATR	[NULL]	2026-04-29 00:00:00.000	2026-05-06 00:00:00.000	[NULL]	[NULL]

Table: order_details(before cancelling order=11087)

	orderid	productid	unitprice	quantity	discount
2132	11,077	3	10	4	0
2133	11,077	4	22	1	0
2134	11,077	6	25	1	0.02
2135	11,077	7	30	1	0.05
2136	11,077	8	40	2	0.1
2137	11,077	10	31	1	0
2138	11,077	12	38	2	0.05
2139	11,077	13	6	4	0
2140	11,077	14	23.25	1	0.03
2141	11,077	16	17.45	2	0.03
2142	11,077	20	81	1	0.04
2143	11,077	23	9	2	0
2144	11,077	32	32	1	0
2145	11,077	39	18	2	0.05
2146	11,077	41	9.65	3	0
2147	11,077	46	12	3	0.02
2148	11,077	52	7	2	0
2149	11,077	55	24	2	0
2150	11,077	60	34	2	0.06
2151	11,077	64	33.25	2	0.03
2152	11,077	66	17	1	0
2153	11,077	73	15	2	0.01
2154	11,077	75	7.75	4	0
2155	11,077	77	13	2	0
2156	11,078	1	18	1	0
2157	11,085	2	19	1	0
2158	11,086	2	19	5	0
2159	11,087	2	19	5	0

Table: orders(after canceling order 11087)

	orderid	customerid	employeeid	orderdate	requireddate	shippeddate	shipvia
802	11,049	GOURL	3	1998-04-24 00:00:00.000	1998-05-22 00:00:00.000	1998-05-04 00:00:00.000	1
803	11,050	FOLKO	8	1998-04-27 00:00:00.000	1998-05-25 00:00:00.000	1998-05-05 00:00:00.000	2
804	11,051	LAMAI	7	1998-04-27 00:00:00.000	1998-05-25 00:00:00.000	[NULL]	3
805	11,052	HANAR	3	1998-04-27 00:00:00.000	1998-05-25 00:00:00.000	1998-05-01 00:00:00.000	1
806	11,053	PICCO	2	1998-04-27 00:00:00.000	1998-05-25 00:00:00.000	1998-04-29 00:00:00.000	2
807	11,054	CACTU	8	1998-04-28 00:00:00.000	1998-05-26 00:00:00.000	[NULL]	1
808	11,055	HILAA	7	1998-04-28 00:00:00.000	1998-05-26 00:00:00.000	1998-05-05 00:00:00.000	2
809	11,056	EASTC	8	1998-04-28 00:00:00.000	1998-05-12 00:00:00.000	1998-05-01 00:00:00.000	2
810	11,057	NORTS	3	1998-04-29 00:00:00.000	1998-05-27 00:00:00.000	1998-05-01 00:00:00.000	3
811	11,058	BLAUS	9	1998-04-29 00:00:00.000	1998-05-27 00:00:00.000	[NULL]	3
812	11,059	RICAR	2	1998-04-29 00:00:00.000	1998-06-10 00:00:00.000	[NULL]	2
813	11,060	FRANS	2	1998-04-30 00:00:00.000	1998-05-28 00:00:00.000	1998-05-04 00:00:00.000	2
814	11,061	GREAL	4	1998-04-30 00:00:00.000	1998-06-11 00:00:00.000	[NULL]	3
815	11,062	REGGC	4	1998-04-30 00:00:00.000	1998-05-28 00:00:00.000	[NULL]	2
816	11,063	HUNGO	3	1998-04-30 00:00:00.000	1998-05-28 00:00:00.000	1998-05-06 00:00:00.000	2
817	11,064	SAVEA	1	1998-05-01 00:00:00.000	1998-05-29 00:00:00.000	1998-05-04 00:00:00.000	1
818	11,065	LILAS	8	1998-05-01 00:00:00.000	1998-05-29 00:00:00.000	[NULL]	1
819	11,066	WHITC	7	1998-05-01 00:00:00.000	1998-05-29 00:00:00.000	1998-05-04 00:00:00.000	2
820	11,067	DRACD	1	1998-05-04 00:00:00.000	1998-05-18 00:00:00.000	1998-05-06 00:00:00.000	2
821	11,068	QUEEN	8	1998-05-04 00:00:00.000	1998-06-01 00:00:00.000	[NULL]	2
822	11,069	TORTU	1	1998-05-04 00:00:00.000	1998-06-01 00:00:00.000	1998-05-06 00:00:00.000	2
823	11,070	LEHMS	2	1998-05-05 00:00:00.000	1998-06-02 00:00:00.000	[NULL]	1
824	11,071	LILAS	1	1998-05-05 00:00:00.000	1998-06-02 00:00:00.000	[NULL]	1
825	11,072	ERNSH	4	1998-05-05 00:00:00.000	1998-06-02 00:00:00.000	[NULL]	2
826	11,073	PERIC	2	1998-05-05 00:00:00.000	1998-06-02 00:00:00.000	[NULL]	2
827	11,074	SIMOB	7	1998-05-06 00:00:00.000	1998-06-03 00:00:00.000	[NULL]	2
828	11,075	RICSU	8	1998-05-06 00:00:00.000	1998-06-03 00:00:00.000	[NULL]	2
829	11,076	BONAP	4	1998-05-06 00:00:00.000	1998-06-03 00:00:00.000	[NULL]	2
830	11,077	RATTC	1	1998-05-06 00:00:00.000	1998-06-03 00:00:00.000	[NULL]	2
831	11,078	ALFKI	5	2026-04-29 00:00:00.000	2026-05-06 00:00:00.000	[NULL]	[NULL]
832	11,085	ANATR	5	2026-04-29 00:00:00.000	2026-05-06 00:00:00.000	[NULL]	[NULL]
833	11,086	ANATR	[NULL]	2026-04-29 00:00:00.000	2026-05-06 00:00:00.000	[NULL]	[NULL]

Refresh ▾ Save ▾ Cancel Export data ▾ 200 833

Table: order_details (after cancelling order=11087)

	orderid	productid	unitprice	quantity	discount
2127	11,075	76	18	2	0.15
2128	11,076	6	25	20	0.25
2129	11,076	14	23.25	20	0.25
2130	11,076	19	9.2	10	0.25
2131	11,077	2	19	24	0.2
2132	11,077	3	10	4	0
2133	11,077	4	22	1	0
2134	11,077	6	25	1	0.02
2135	11,077	7	30	1	0.05
2136	11,077	8	40	2	0.1
2137	11,077	10	31	1	0
2138	11,077	12	38	2	0.05
2139	11,077	13	6	4	0
2140	11,077	14	23.25	1	0.03
2141	11,077	16	17.45	2	0.03
2142	11,077	20	81	1	0.04
2143	11,077	23	9	2	0
2144	11,077	32	32	1	0
2145	11,077	39	18	2	0.05
2146	11,077	41	9.65	3	0
2147	11,077	46	12	3	0.02
2148	11,077	52	7	2	0
2149	11,077	55	24	2	0
2150	11,077	60	34	2	0.06
2151	11,077	64	33.25	2	0.03
2152	11,077	66	17	1	0
2153	11,077	73	15	2	0.01
2154	11,077	75	7.75	4	0
2155	11,077	77	13	2	0
2156	11,078	1	18	1	0
2157	11,085	2	19	1	0
2158	11,086	2	19	5	0

Refresh ▾ Save ▾ Cancel Export data ▾ 200 2,158

2158 row(s) fetched - 0.004s, on 2026-04-29 at 19:48:26

5.Create a stored procedure to transfer products from one supplier to another. If the old supplier or new supplier does not exist, rollback.

```
create or replace procedure transfer_products(
p_old_supplierid int,
p_new_supplierid int
)
language plpgsql
as $$
begin
        if not exists (select 1 from suppliers where supplierid =p_old_supplierid)
then
        raise exception 'old supplier % does not exist', p_old_supplierid;
rollback;
end if;

if not exists (select 1 from suppliers where supplierid = p_new_supplierid) then
raise exception 'new supplier % not exist', p_new_supplierid;
rollback;
end if;

update products
set supplierid = p_new_supplierid
where supplierid = p_old_supplierid;

commit;

raise notice 'products transferred from supplier % to supplier %', p_old_supplierid,
p_new_supplierid;
end;
$$;


call transfer_products(1, 2);
call transfer_products(2, 10);
call transfer_products(3, 6);
call transfer_products(6, 12);
```

outputs:

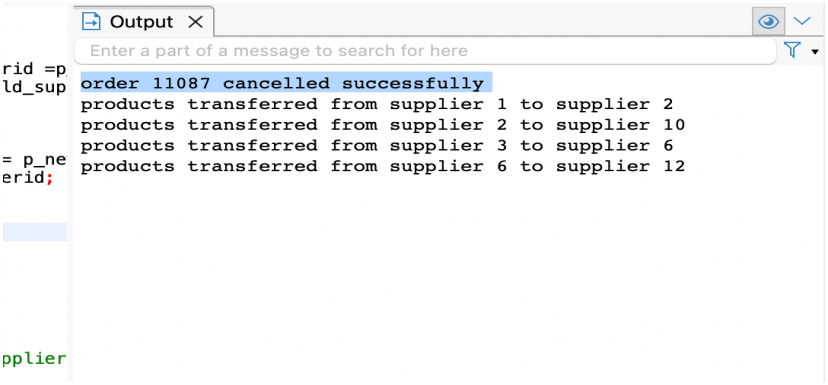
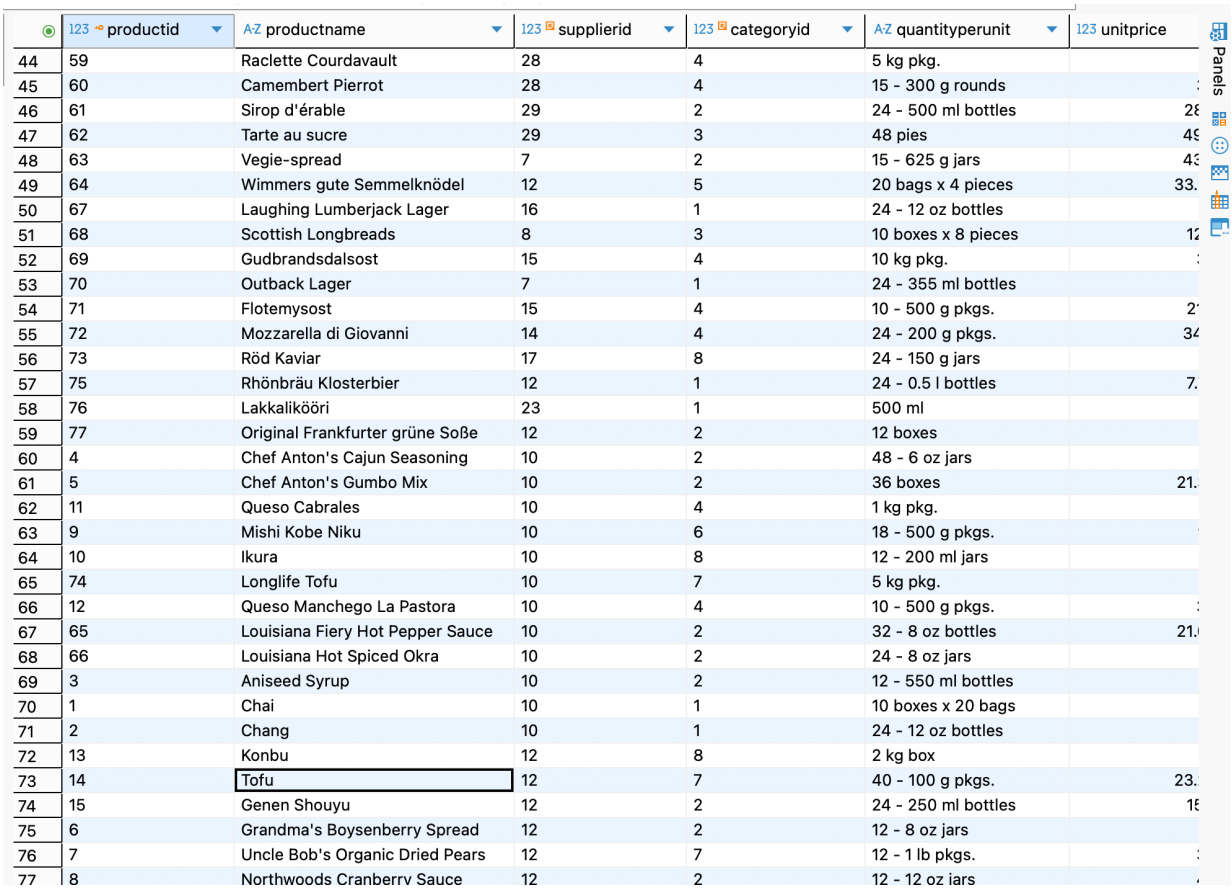


table: products (before changing suppliers)

	productid	productnar	sup	ca	quantityperuni	unitprice	unitsinstock	unitsonorder	reorderlevel
50	52	Filo Mix	24	5	16 - 2 kg boxes	7	38	0	2
51	53	Perth Pasties	24	6	48 pieces	32.8	0	0	
52	54	Tourtière	25	6	16 pies	7.45	21	0	1
53	55	Pâté chinois	25	6	24 boxes x 2 pies	24	115	0	2
54	56	Gnocchi di nonna	26	5	24 - 250 g pkgs.	38	21	10	3
55	57	Ravioli Angelo	26	5	24 - 250 g pkgs.	19.5	36	0	2
56	58	Escargots de Bour	27	8	24 pieces	13.25	62	0	2
57	59	Raclette Courdava	28	4	5 kg pkg.	55	79	0	
58	60	Camembert Pierro	28	4	15 - 300 g rounds	34	19	0	
59	61	Sirop d'érable	29	2	24 - 500 ml bottles	28.5	113	0	2
60	62	Tarte au sucre	29	3	48 pies	49.3	17	0	
61	63	Vegie-spread	7	2	15 - 625 g jars	43.9	24	0	
62	64	Wimmers gute Sei	12	5	20 bags x 4 pieces	33.25	22	80	3
63	65	Louisiana Fiery Hc	2	2	32 - 8 oz bottles	21.05	76	0	
64	66	Louisiana Hot Spic	2	2	24 - 8 oz jars	17	4	100	2
65	67	Laughing Lumberj	16	1	24 - 12 oz bottles	14	52	0	1
66	68	Scottish Longbrea	8	3	10 boxes x 8 pieces	12.5	6	10	
67	69	Gudbrandsdalsost	15	4	10 kg pkg.	36	26	0	
68	70	Outback Lager	7	1	24 - 355 ml bottles	15	15	10	3
69	71	Flotemysost	15	4	10 - 500 g pkgs.	21.5	26	0	
70	72	Mozzarella di Giov	14	4	24 - 200 g pkgs.	34.8	14	0	
71	73	Röd Kaviar	17	8	24 - 150 g jars	15	101	0	
72	74	Longlife Tofu	4	7	5 kg pkg.	10	4	20	
73	75	Rhönbräu Klosterb	12	1	24 - 0.5 l bottles	7.75	125	0	2
74	76	Lakkalikööri	23	1	500 ml	18	57	0	2
75	77	Original Frankfurte	12	2	12 boxes	13	32	0	
76	1	Chai	1	1	10 boxes x 20 bags	18	38	0	1
77	2	Chang	1	1	24 - 12 oz bottles	19	6	40	2

table: products (after changing suppliers)



	productid	productname	supplierid	categoryid	quantityperunit	unitprice
44	59	Raclette Courdavault	28	4	5 kg pkg.	
45	60	Camembert Pierrot	28	4	15 - 300 g rounds	
46	61	Sirop d'érable	29	2	24 - 500 ml bottles	28
47	62	Tarte au sucre	29	3	48 pies	49
48	63	Vegie-spread	7	2	15 - 625 g jars	43
49	64	Wimmers gute Semmelknödel	12	5	20 bags x 4 pieces	33.
50	67	Laughing Lumberjack Lager	16	1	24 - 12 oz bottles	
51	68	Scottish Longbreads	8	3	10 boxes x 8 pieces	12
52	69	Gudbrandsdalsost	15	4	10 kg pkg.	
53	70	Outback Lager	7	1	24 - 355 ml bottles	
54	71	Flotemysost	15	4	10 - 500 g pkgs.	2
55	72	Mozzarella di Giovanni	14	4	24 - 200 g pkgs.	34
56	73	Röd Kaviar	17	8	24 - 150 g jars	
57	75	Rhönbräu Klosterbier	12	1	24 - 0.5 l bottles	7.
58	76	Lakkalikööri	23	1	500 ml	
59	77	Original Frankfurter grüne Soße	12	2	12 boxes	
60	4	Chef Anton's Cajun Seasoning	10	2	48 - 6 oz jars	
61	5	Chef Anton's Gumbo Mix	10	2	36 boxes	21.
62	11	Queso Cabrales	10	4	1 kg pkg.	
63	9	Mishi Kobe Niku	10	6	18 - 500 g pkgs.	
64	10	Ikura	10	8	12 - 200 ml jars	
65	74	Longlife Tofu	10	7	5 kg pkg.	
66	12	Queso Manchego La Pastora	10	4	10 - 500 g pkgs.	
67	65	Louisiana Fiery Hot Pepper Sauce	10	2	32 - 8 oz bottles	21.
68	66	Louisiana Hot Spiced Okra	10	2	24 - 8 oz jars	
69	3	Aniseed Syrup	10	2	12 - 550 ml bottles	
70	1	Chai	10	1	10 boxes x 20 bags	
71	2	Chang	10	1	24 - 12 oz bottles	
72	13	Konbu	12	8	2 kg box	
73	14	Tofu	12	7	40 - 100 g pkgs.	23.
74	15	Genen Shouyu	12	2	24 - 250 ml bottles	15
75	6	Grandma's Boysenberry Spread	12	2	12 - 8 oz jars	
76	7	Uncle Bob's Organic Dried Pears	12	7	12 - 1 lb pkgs.	
77	8	Northwoods Cranberry Sauce	12	2	12 - 12 oz jars	

6. Create a stored procedure to update the price of all products in a category by a percentage. Rollback if the percentage is less than or equal to zero.

```
create or replace procedure update_category_prices(
p_categoryid int,
p_percentage decimal
)
language plpgsql
as $$
begin
        if p_percentage <= 0 then
        raise exception 'percentage must be greater than 0';
rollback;
end if;
```

```
if not exists (select 1 from categories where categoryid = p_categoryid ) then
raise exception 'category % does not exist', p_categoryid;
rollback;
end if;

update products
set unitprice = unitprice * (1+p_percentage / 100)
where categoryid = p_categoryid;

commit;
raise notice 'prices updated for category % by % percentage', p_categoryid,
p_percentage;
end;
$$;

call update_category_prices(3, 10);
call update_category_prices(6, 12);
```

output:

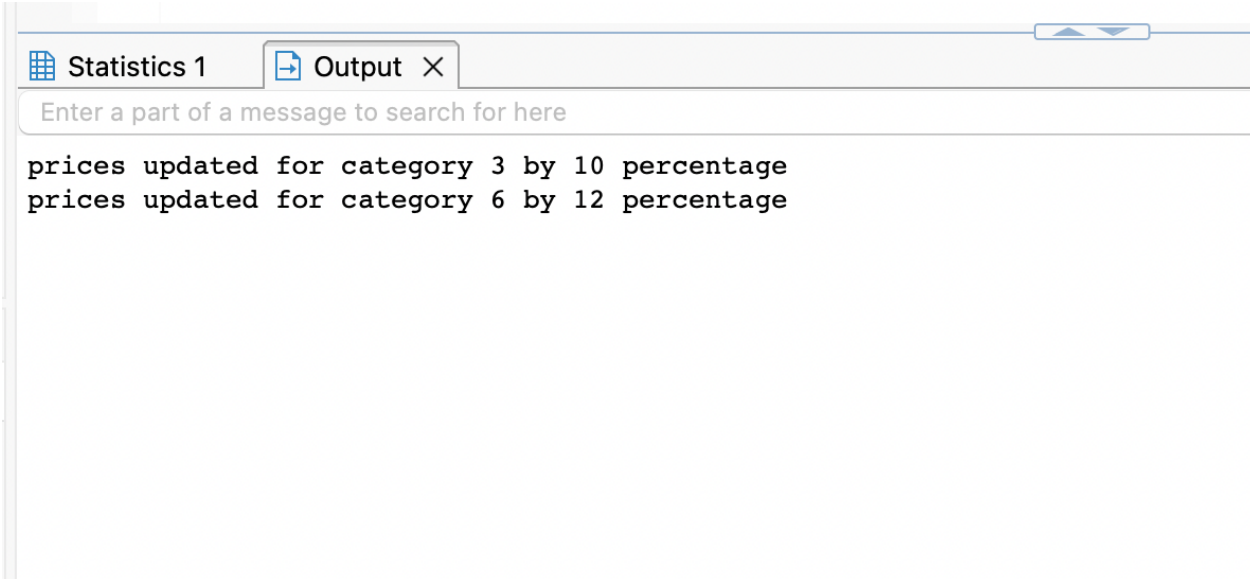
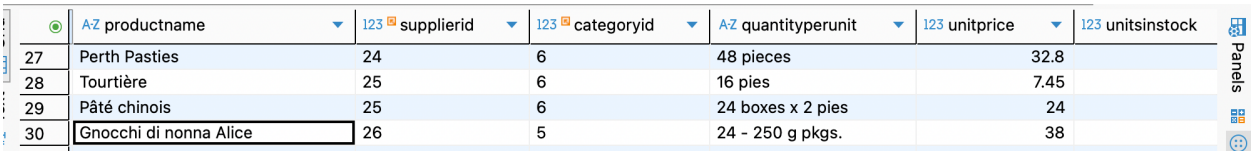
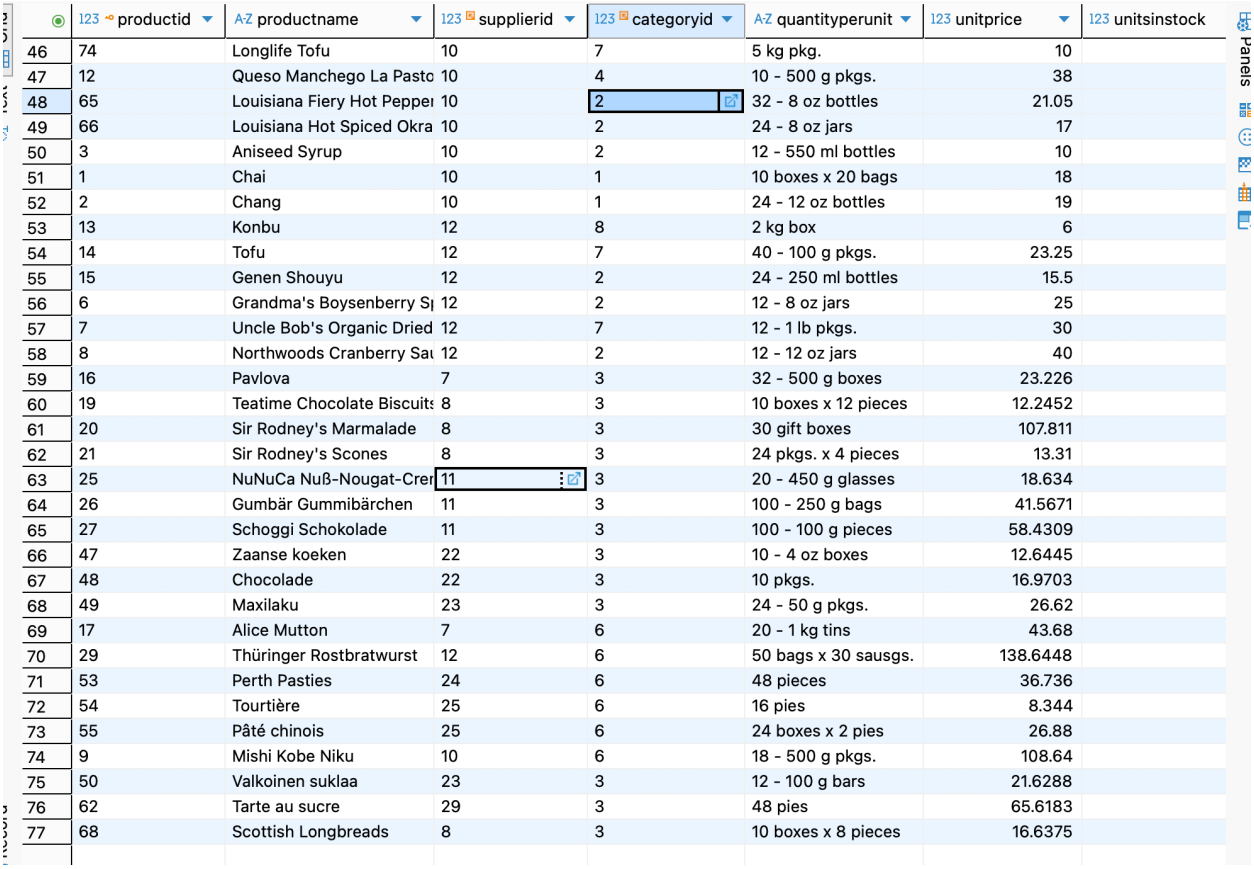


Table: products(before updating prices)



	productname	supplierid	categoryid	quantityperunit	unitprice	unitsinstock
27	Perth Pasties	24	6	48 pieces	32.8	
28	Tourtière	25	6	16 pies	7.45	
29	Pâté chinois	25	6	24 boxes x 2 pies	24	
30	Gnocchi di nonna Alice	26	5	24 - 250 g pkgs.	38	

Table: products (after updating prices by given percentage)



	productid	productname	supplierid	categoryid	quantityperunit	unitprice	unitsinstock
46	74	Longlife Tofu	10	7	5 kg pkg.	10	
47	12	Queso Manchego La Pasto	10	4	10 - 500 g pkgs.	38	
48	65	Louisiana Fiery Hot Pepper	10	2	32 - 8 oz bottles	21.05	
49	66	Louisiana Hot Spiced Okra	10	2	24 - 8 oz jars	17	
50	3	Aniseed Syrup	10	2	12 - 550 ml bottles	10	
51	1	Chai	10	1	10 boxes x 20 bags	18	
52	2	Chang	10	1	24 - 12 oz bottles	19	
53	13	Konbu	12	8	2 kg box	6	
54	14	Tofu	12	7	40 - 100 g pkgs.	23.25	
55	15	Genen Shouyu	12	2	24 - 250 ml bottles	15.5	
56	6	Grandma's Boysenberry Sp	12	2	12 - 8 oz jars	25	
57	7	Uncle Bob's Organic Dried	12	7	12 - 1 lb pkgs.	30	
58	8	Northwoods Cranberry Sau	12	2	12 - 12 oz jars	40	
59	16	Pavlova	7	3	32 - 500 g boxes	23.226	
60	19	Teatime Chocolate Biscuits	8	3	10 boxes x 12 pieces	12.2452	
61	20	Sir Rodney's Marmalade	8	3	30 gift boxes	107.811	
62	21	Sir Rodney's Scones	8	3	24 pkgs. x 4 pieces	13.31	
63	25	NuNuCa Nuß-Nougat-Crem	11	3	20 - 450 g glasses	18.634	
64	26	Gumbär Gummibärchen	11	3	100 - 250 g bags	41.5671	
65	27	Schoggi Schokolade	11	3	100 - 100 g pieces	58.4309	
66	47	Zaanse koeken	22	3	10 - 4 oz boxes	12.6445	
67	48	Chocolade	22	3	10 pkgs.	16.9703	
68	49	Maxilaku	23	3	24 - 50 g pkgs.	26.62	
69	17	Alice Mutton	7	6	20 - 1 kg tins	43.68	
70	29	Thüringer Rostbratwurst	12	6	50 bags x 30 sausgs.	138.6448	
71	53	Perth Pasties	24	6	48 pieces	36.736	
72	54	Tourtière	25	6	16 pies	8.344	
73	55	Pâté chinois	25	6	24 boxes x 2 pies	26.88	
74	9	Mishi Kobe Niku	10	6	18 - 500 g pkgs.	108.64	
75	50	Valkoinen suklaa	23	3	12 - 100 g bars	21.6288	
76	62	Tarte au sucre	29	3	48 pies	65.6183	
77	68	Scottish Longbreads	8	3	10 boxes x 8 pieces	16.6375	

7. Create a stored procedure to add a new product under an existing category and supplier. Rollback if the category or supplier does not exist.

```
create or replace  procedure add_new_product(
p_productname varchar(40),
p_categoryid int,
```

```
p_supplierid int,
p_unitprice decimal
)
language plpgsql
as $$
begin
	if not exists (select 1 from categories where categoryid = p_categoryid)
then
	raise exception 'category % does not exist', p_categoryid;
rollback;
end if;

if not exists (select 1 from suppliers where supplierid = p_supplierid) then
raise exception 'supplier % does not exist', p_supplierid;
rollback;
end if;

insert into products (productname, categoryid, supplierid, unitprice)
values (p_productname, p_categoryid, p_supplierid, p_unitprice);
commit;
raise notice 'product % added successfully under category % and supplier %',
p_productname, p_categoryid, p_supplierid;
end;
$$;

call add_new_product('new product', 1, 1, 25.78);
```

output:

product new product added successfully under category 1 and supplier 1

Table: products with new product (productids = 78,79)

	productid	productname	supplierid	categoryid	quantityperunit	unitprice
48	65	Louisiana Fiery Hot Pepper Sauce	10	2	32 - 8 oz bottles	21.
49	66	Louisiana Hot Spiced Okra	10	2	24 - 8 oz jars	
50	3	Aniseed Syrup	10	2	12 - 550 ml bottles	
51	1	Chai	10	1	10 boxes x 20 bags	
52	2	Chang	10	1	24 - 12 oz bottles	
53	13	Konbu	12	8	2 kg box	
54	14	Tofu	12	7	40 - 100 g pkgs.	23.
55	15	Genen Shouyu	12	2	24 - 250 ml bottles	15
56	6	Grandma's Boysenberry Spread	12	2	12 - 8 oz jars	
57	7	Uncle Bob's Organic Dried Pears	12	7	12 - 1 lb pkgs.	
58	8	Northwoods Cranberry Sauce	12	2	12 - 12 oz jars	
59	16	Pavlova	7	3	32 - 500 g boxes	23.2
60	19	Teatime Chocolate Biscuits	8	3	10 boxes x 12 pieces	12.24
61	20	Sir Rodney's Marmalade	8	3	30 gift boxes	107.8
62	21	Sir Rodney's Scones	8	3	24 pkgs. x 4 pieces	13.
63	25	NuNuCa Nuß-Nougat-Creme	11	3	20 - 450 g glasses	18.6
64	26	Gumbär Gummibärchen	11	3	100 - 250 g bags	41.56
65	27	Schoggi Schokolade	11	3	100 - 100 g pieces	58.43
66	47	Zaanse koeken	22	3	10 - 4 oz boxes	12.64
67	48	Chocolade	22	3	10 pkgs.	16.97
68	49	Maxilaku	23	3	24 - 50 g pkgs.	26.
69	17	Alice Mutton	7	6	20 - 1 kg tins	43.
70	29	Thüringer Rostbratwurst	12	6	50 bags x 30 sausgs.	138.64
71	53	Perth Pasties	24	6	48 pieces	36.7
72	54	Tourtière	25	6	16 pies	8.3
73	55	Pâté chinois	25	6	24 boxes x 2 pies	26.
74	9	Mishi Kobe Niku	10	6	18 - 500 g pkgs.	108.
75	50	Valkoinen suklaa	23	3	12 - 100 g bars	21.62
76	62	Tarte au sucre	29	3	48 pies	65.61
77	68	Scottish Longbreads	8	3	10 boxes x 8 pieces	16.63
78	78	new product	1	1	[NULL]	25.
79	79	new product	1	1	[NULL]	25.

8. Create a stored procedure to delete a customer only if the customer has no orders. Use a transaction and rollback if orders exist.

```
create or replace procedure delete_customer(
p_customerid varchar(5)
)
language plpgsql
as $$
begin
        if not exists (select 1 from customers where customerid = p_customerid)
then
        raise exception 'customer % does not exist', p_customerid;
rollback;
end if;

        if exists (select 1 from orders where customerid = p_customerid) then
```

```
        raise exception 'customer % has orders and cannot be deleted',
p_customerid;
rollback;
end if;

delete from customers where customerid = p_customerid;

commit;
raise notice 'customer % deleted successfully', p_customerid;
end;
$$;

call delete_customer('AS002');
call delete_customer('PARIS');
call delete_customer('AS001');
```

OUTPUT:

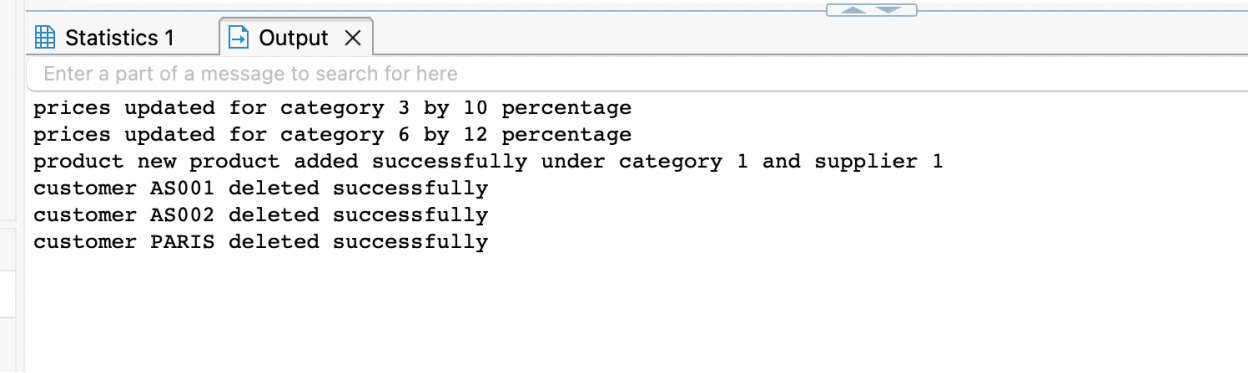
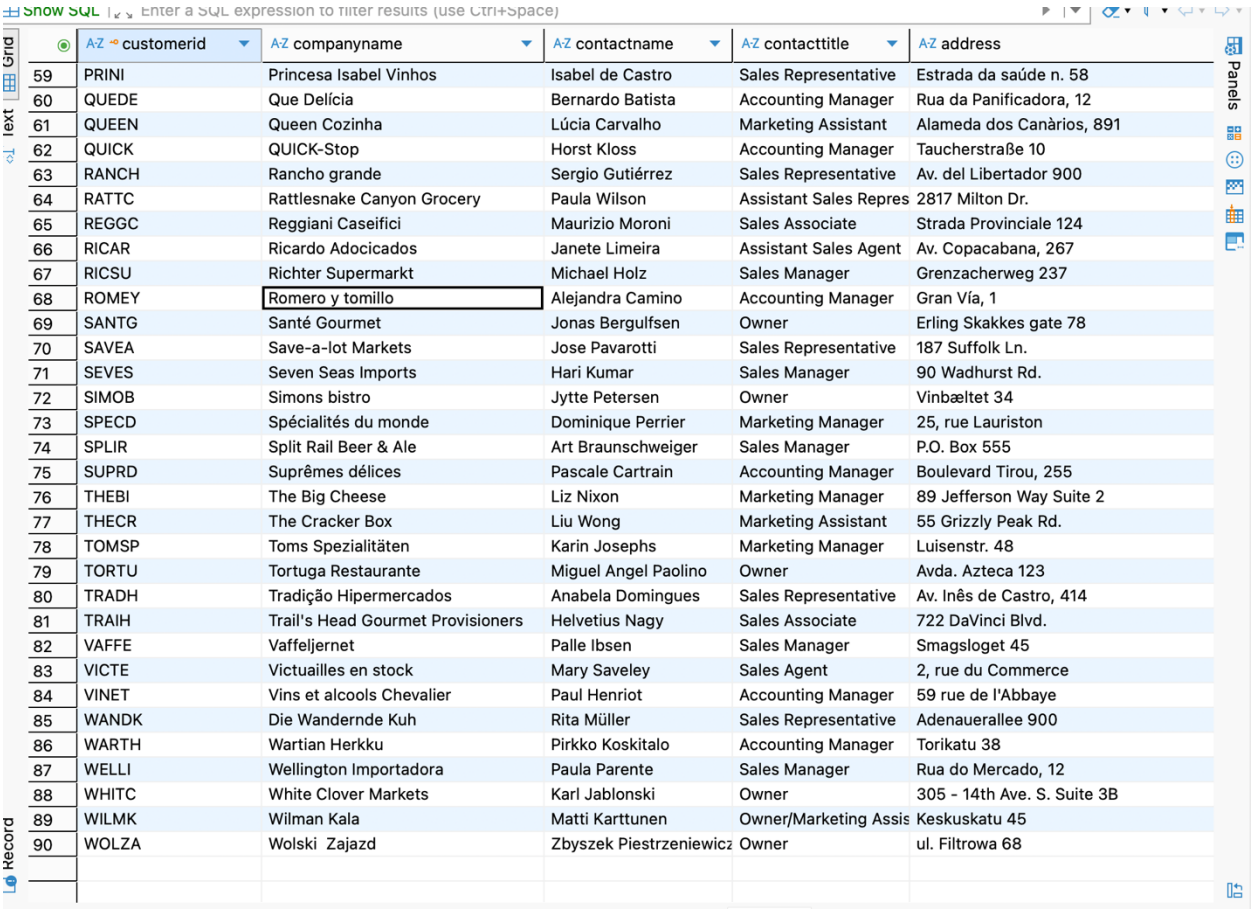


Table: customers (before deleting customers who has 0 orders)

	customerid	companyname	contactname	contacttitle	address	city	region
62	QUEEN	Queen Cozinha	Lúcia Carvalho	Marketing Assistant	Alameda dos Canài	Sao Paulo	SP
63	QUICK	QUICK-Stop	Horst Kloss	Accounting Manager	Taucherstraße 10	Cunewalde	[NULL]
64	RANCH	Rancho grande	Sergio Gutiérrez	Sales Representative	Av. del Libertador 9	Buenos Aires	[NULL]
65	RATTC	Rattlesnake Canyon Groc	Paula Wilson	Assistant Sales Repres	2817 Milton Dr.	Albuquerque	NM
66	REGGC	Reggiani Caseifici	Maurizio Moroni	Sales Associate	Strada Provinciale '	Reggio Emilia	[NULL]
67	RICAR	Ricardo Adocicados	Janete Limeira	Assistant Sales Agent	Av. Copacabana, 24	Rio de Janeiro	RJ
68	RICSU	Richter Supermarkt	Michael Holz	Sales Manager	Grenzacherweg 23	Genève	[NULL]
69	ROMEY	Romero y tomillo	Alejandra Camino	Accounting Manager	Gran Vía, 1	Madrid	[NULL]
70	SANTG	Santé Gourmet	Jonas Bergulfsen	Owner	Erling Skakkes gate	Stavern	[NULL]
71	SAVEA	Save-a-lot Markets	Jose Pavarotti	Sales Representative	187 Suffolk Ln.	Boise	ID
72	SEVES	Seven Seas Imports	Hari Kumar	Sales Manager	90 Wadhurst Rd.	London	[NULL]
73	SIMOB	Simons bistro	Jytte Petersen	Owner	Vinbæltet 34	Kobenhavn	[NULL]
74	SPECD	Spécialités du monde	Dominique Perrier	Marketing Manager	25, rue Lauriston	Paris	[NULL]
75	SPLIR	Split Rail Beer & Ale	Art Braunschweiger	Sales Manager	P.O. Box 555	Lander	WY
76	SUPRD	Suprêmes délices	Pascale Cartrain	Accounting Manager	Boulevard Tirou, 25	Charleroi	[NULL]
77	THEBI	The Big Cheese	Liz Nixon	Marketing Manager	89 Jefferson Way S	Portland	OR
78	THECR	The Cracker Box	Liu Wong	Marketing Assistant	55 Grizzly Peak Rd	Butte	MT
79	TOMSP	Toms Spezialitäten	Karin Josephs	Marketing Manager	Luisenstr. 48	Münster	[NULL]
80	TORTU	Tortuga Restaurante	Miguel Angel Paolino	Owner	Avda. Azteca 123	México D.F.	[NULL]
81	TRADH	Tradição Hipermercados	Anabela Domingues	Sales Representative	Av. Inês de Castro,	Sao Paulo	SP
82	TRAIH	Trail's Head Gourmet Prov	Helvetius Nagy	Sales Associate	722 DaVinci Blvd.	Kirkland	WA
83	VAFFE	Vaffeljernet	Palle Ibsen	Sales Manager	Smagsloget 45	Århus	[NULL]
84	VICTE	Victuailles en stock	Mary Saveley	Sales Agent	2, rue du Commerc	Lyon	[NULL]
85	VINET	Vins et alcools Chevalier	Paul Henriot	Accounting Manager	59 rue de l'Abbaye	Reims	[NULL]
86	WANDK	Die Wandernde Kuh	Rita Müller	Sales Representative	Adenauerallee 900	Stuttgart	[NULL]
87	WARTH	Wartian Herkku	Pirkko Koskitalo	Accounting Manager	Torikatu 38	Oulu	[NULL]
88	WELLI	Wellington Importadora	Paula Parente	Sales Manager	Rua do Mercado, 12	Resende	SP
89	WHITC	White Clover Markets	Karl Jablonski	Owner	305 - 14th Ave. S. S	Seattle	WA
90	WILMK	Wilman Kala	Matti Karttunen	Owner/Marketing Assis	Keskuskatu 45	Helsinki	[NULL]
91	WOLZA	Wolski Zajazd	Zbyszek Piestrzeniewicz	Owner	ul. Filtrowa 68	Warszawa	[NULL]
92	AS001	Alia	ABC Corp	[NULL]	[NULL]	[NULL]	[NULL]
93	AS002	Blia	ABC Corp	[NULL]	[NULL]	[NULL]	[NULL]

Table: customers (after deleting customers who has 0 orders)



	customerid	companyname	contactname	contacttitle	address
59	PRINI	Princesa Isabel Vinhos	Isabel de Castro	Sales Representative	Estrada da saúde n. 58
60	QUEDE	Que Delícia	Bernardo Batista	Accounting Manager	Rua da Panificadora, 12
61	QUEEN	Queen Cozinha	Lúcia Carvalho	Marketing Assistant	Alameda dos Canàrios, 891
62	QUICK	QUICK-Stop	Horst Kloss	Accounting Manager	Taucherstraße 10
63	RANCH	Rancho grande	Sergio Gutiérrez	Sales Representative	Av. del Libertador 900
64	RATTC	Rattlesnake Canyon Grocery	Paula Wilson	Assistant Sales Repres	2817 Milton Dr.
65	REGGC	Reggiani Caseifici	Maurizio Moroni	Sales Associate	Strada Provinciale 124
66	RICAR	Ricardo Adocicados	Janete Limeira	Assistant Sales Agent	Av. Copacabana, 267
67	RICSU	Richter Supermarkt	Michael Holz	Sales Manager	Grenzacherweg 237
68	ROMEY	Romero y tomillo	Alejandra Camino	Accounting Manager	Gran Vía, 1
69	SANTG	Santé Gourmet	Jonas Bergulfsen	Owner	Erling Skakkes gate 78
70	SAVEA	Save-a-lot Markets	Jose Pavarotti	Sales Representative	187 Suffolk Ln.
71	SEVES	Seven Seas Imports	Hari Kumar	Sales Manager	90 Wadhurst Rd.
72	SIMOB	Simons bistro	Jytte Petersen	Owner	Vinbæltet 34
73	SPECD	Spécialités du monde	Dominique Perrier	Marketing Manager	25, rue Lauriston
74	SPLIR	Split Rail Beer & Ale	Art Braunschweiger	Sales Manager	P.O. Box 555
75	SUPRD	Suprêmes délices	Pascale Cartrain	Accounting Manager	Boulevard Tirou, 255
76	THEBI	The Big Cheese	Liz Nixon	Marketing Manager	89 Jefferson Way Suite 2
77	THECR	The Cracker Box	Liu Wong	Marketing Assistant	55 Grizzly Peak Rd.
78	TOMSP	Toms Spezialitäten	Karin Josephs	Marketing Manager	Luisenstr. 48
79	TORTU	Tortuga Restaurante	Miguel Angel Paolino	Owner	Avda. Azteca 123
80	TRADH	Tradição Hipermercados	Anabela Domingues	Sales Representative	Av. Inês de Castro, 414
81	TRAIH	Trail's Head Gourmet Provisioners	Helvetius Nagy	Sales Associate	722 DaVinci Blvd.
82	VAFFE	Vaffeljernet	Palle Ibsen	Sales Manager	Smagsloget 45
83	VICTE	Victuailles en stock	Mary Saveley	Sales Agent	2, rue du Commerce
84	VINET	Vins et alcools Chevalier	Paul Henriot	Accounting Manager	59 rue de l'Abbaye
85	WANDK	Die Wandernde Kuh	Rita Müller	Sales Representative	Adenauerallee 900
86	WARTH	Wartian Herkku	Pirkko Koskitalo	Accounting Manager	Torikatu 38
87	WELLI	Wellington Importadora	Paula Parente	Sales Manager	Rua do Mercado, 12
88	WHITC	White Clover Markets	Karl Jablonski	Owner	305 - 14th Ave. S. Suite 3B
89	WILMK	Wilman Kala	Matti Karttunen	Owner/Marketing Assis	Keskuskatu 45
90	WOLZA	Wolski Zajazd	Zbyszek Piestrzeniewicz	Owner	ul. Filtrowa 68

9. Create a stored procedure to apply a discount to all order details for a specific order. Rollback if the order does not exist.

```
create or replace procedure apply_discount(
p_orderid int,
p_discount decimal
)
language plpgsql
as $$
begin
        if not exists (select 1 from orders where orderid = p_orderid) then
        raise exception 'order % does not exist', p_orderid;
rollback;
end if;
```

```
if p_discount < 0 or p_discount > 1 then
raise exception 'discount must be between 0 and 1';
rollback;
end if;

update order_details
set discount = p_discount
where orderid = p_orderid;

commit;
raise notice 'discount of % applied to all products in order %', p_discount,
p_orderid;

end;
$$;

call apply_discount(10250, 0.10);
```

output:

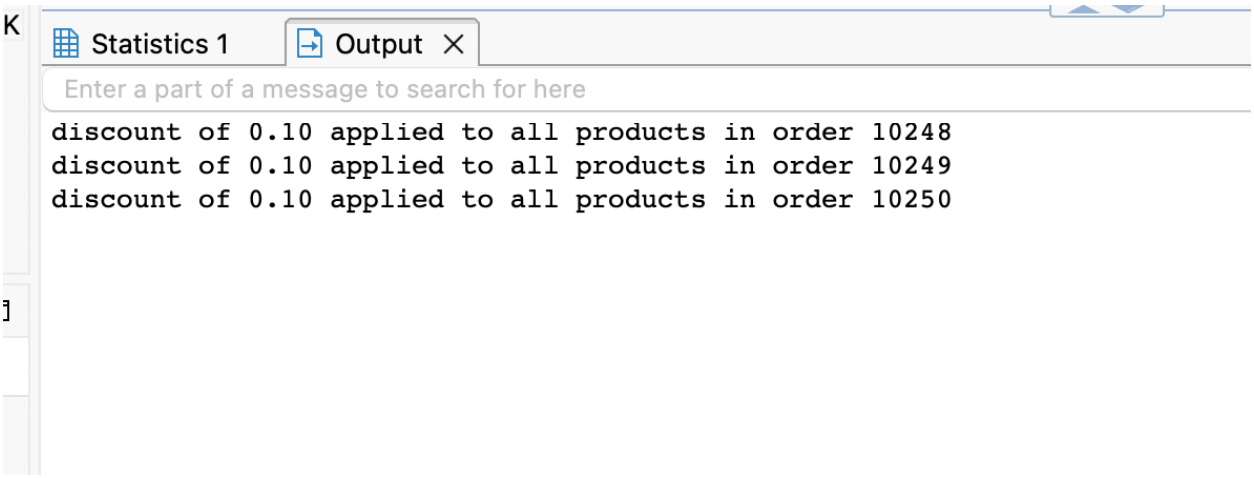


Table: order details(before applying discount for products on ordered=10248 10249)

	orderid	productid	unitprice	quantity	discount
1	10,249	14	18.6	9	0
2	10,249	51	42.4	40	0
3	10,250	41	7.7	10	0
4	10,250	51	42.4	35	0.15
5	10,250	65	16.8	15	0.15
6	10,251	22	16.8	6	0.05
7	10,251	57	15.6	15	0.05
8	10,251	65	16.8	20	0
9	10,252	20	64.8	40	0.05
10	10,252	33	2	25	0.05

Table: order details(after applying discount for products on ordered=10248)

	orderid	productid	unitprice	quantity	discount
2127	11,076	19	9.2	10	0.25
2128	11,077	2	19	24	0.2
2129	11,077	3	10	4	0
2130	11,077	4	22	1	0
2131	11,077	6	25	1	0.02
2132	11,077	7	30	1	0.05
2133	11,077	8	40	2	0.1
2134	11,077	10	31	1	0
2135	11,077	12	38	2	0.05
2136	11,077	13	6	4	0
2137	11,077	14	23.25	1	0.03
2138	11,077	16	17.45	2	0.03
2139	11,077	20	81	1	0.04
2140	11,077	23	9	2	0
2141	11,077	32	32	1	0
2142	11,077	39	18	2	0.05
2143	11,077	41	9.65	3	0
2144	11,077	46	12	3	0.02
2145	11,077	52	7	2	0
2146	11,077	55	24	2	0
2147	11,077	60	34	2	0.06
2148	11,077	64	33.25	2	0.03
2149	11,077	66	17	1	0
2150	11,077	73	15	2	0.01
2151	11,077	75	7.75	4	0
2152	11,077	77	13	2	0
2153	11,078	1	18	1	0
2154	11,085	2	19	1	0
2155	11,086	2	19	5	0
2156	10,248	11	14	12	0.1
2157	10,248	42	9.8	10	0.1
2158	10,248	72	34.8	5	0.1

10. Create a stored procedure to place an order with multiple products. Insert the order and all order items in one transaction. If any product is invalid or stock is insufficient, rollback the complete order.

```
 create or replace procedure place_multiple_order(
    p_customerid varchar(5),
    p_employeeid int,
    p_products jsonb
)
language plpgsql
as $$
declare
    v_orderid int;
    v_product record;
    v_unitprice decimal;
    v_product_count int;
begin
    if not exists (select 1 from customers where customerid = p_customerid) then
        raise exception 'customer % does not exist', p_customerid;
    end if;

    insert into orders (customerid, employeeid, orderdate)
    values (p_customerid, p_employeeid, current_date)
    returning orderid into v_orderid;

    v_product_count := 0;

    for v_product in
        select * from jsonb_to_recordset(p_products) as x(productid int, quantity
int)
    loop
        select unitprice into v_unitprice
        from products
        where productid = v_product.productid;

        if not found then
            raise exception 'product % does not exist', v_product.productid;
        end if;
```

```
        if (select unitsinstock from products where productid = v_product.productid)
< v_product.quantity then
            raise exception 'insufficient stock for product %', v_product.productid;
        end if;

        insert into order_details(orderid, productid, unitprice, quantity)
        values (v_orderid, v_product.productid, v_unitprice, v_product.quantity);

        update products
        set unitsinstock = unitsinstock - v_product.quantity
        where productid = v_product.productid;

        v_product_count := v_product_count + 1;
    end loop;

    commit;
    raise notice 'order % placed successfully with % products', v_orderid,
v_product_count;

end;
$$;

call place_multiple_order('ANATR', 2, '[{"productid": 18, "quantity": 5},
{"productid": 22, "quantity": 5}]'::jsonb);
```

output:
order 11091 placed successfully with 2 products

Table: products before placing orders on product ids 18 and 22 stocks are 42 and 104

	productid	productname	supp	cate	quantityperur	unitpr	unitsinstock	unitsonorder
1	18	Carnarvon Tigers	7	8	16 kg pkg.	62.5	42	
2	22	Gustaf's Knäckebröd	9	5	24 - 500 g pkgs.	21	104	
3	23	Tunnbröd	9	5	12 - 250 g pkgs.	9	61	
4	24	Guaraná Fantástica	10	1	12 - 355 ml cans	4.5	20	
5	28	Rössle Sauerkraut	12	7	25 - 825 g cans	45.6	26	
6	30	Nord-Ost Matjeshering	13	8	10 - 200 g glasses	25.89	10	
7	31	Gorgonzola Telino	14	4	12 - 100 g pkgs	12.5	0	

Table: order_details

	orderid	productid	unitprice	quantity	discount
2131	11,077	7	30	1	0.05
2132	11,077	8	40	2	0.1
2133	11,077	10	31	1	0
2134	11,077	12	38	2	0.05
2135	11,077	13	6	4	0
2136	11,077	14	23.25	1	0.03
2137	11,077	16	17.45	2	0.03
2138	11,077	20	81	1	0.04
2139	11,077	23	9	2	0
2140	11,077	32	32	1	0
2141	11,077	39	18	2	0.05
2142	11,077	41	9.65	3	0
2143	11,077	46	12	3	0.02
2144	11,077	52	7	2	0
2145	11,077	55	24	2	0
2146	11,077	60	34	2	0.06
2147	11,077	64	33.25	2	0.03
2148	11,077	66	17	1	0
2149	11,077	73	15	2	0.01
2150	11,077	75	7.75	4	0
2151	11,077	77	13	2	0
2152	11,078	1	18	1	0
2153	11,085	2	19	1	0
2154	11,086	2	19	5	0
2155	10,248	11	14	12	0.1
2156	10,248	42	9.8	10	0.1
2157	10,248	72	34.8	5	0.1
2158	10,250	65	16.8	15	0.1
2159	11,088	1	18	5	0
2160	11,088	2	19	6	0
2161	11,091	18	62.5	5	0
2162	11,091	22	21	5	0

Table: products with stock updation after placing orders on product ids 18 and 22, stocks are 37 and 99.